```
(*
                              CS51 Lab 18
                          Environment Semantics
 *)
(*
                                SOLUTION
 *)

(*======================================================================
Readings:

    This lab builds on material from Chapter 19 of the textbook
    <http://book.cs51.io>, which should be read before the lab session.

Objective:

    This lab practices concepts of environment semantics. You'll carry out
    derivations using the various rule sets from Chapter 19, and gain
    intuition about dynamic versus lexical semantics and how stores work
    to allow mutability. The payoff exercises here are Exercises 9, 11,
    and 12.

    Finally, you'll program a simple implementation of environments --
    allowing lookup in and extension of environments -- which may be
    helpful with your work on the final project.
======================================================================*)
(*======================================================================
  Part 1: An environment semantics derivation

In this part, you'll work out the formal derivation of the environment
semantics for the expression

    let x = 3 + 5 in
    (fun x -> x * x) (x - 2)

according to the semantic rules presented in Chapter 19, Figure 19.1,
just as you did in Lab 9, Part 1 for substitution semantics.

Before beginning, what should this expression evaluate to? Test out
your prediction in the OCaml REPL. *)

(* ANSWER: The expression evaluates to 36:

    # let x = 3 + 5 in
        (fun x -> x * x) (x - 2)  ;;
    - : int = 36
*)

(* The exercises will take you through the derivation stepwise, so
that you can use the results from earlier exercises in the later
exercises.

```

```
By way of example, we do the first three exercises for you to give
you the idea.

.....................................................................
Exercise 1. Carry out the derivation for the semantics of the
expression `3 + 5` in an empty environment.
.....................................................................*)

(* ANSWER:

    {} ⊢ 3 + 5 ⇓
               | {} ⊢ 3 ⇓ 3       (R_int)
               | {} ⊢ 5 ⇓ 5       (R_int)
               ⇓ 8                (R_+)
*)

(*.....................................................................
Exercise 2. Determine the result of evaluating the following
expression `x + 5` in the environment `{x ↦ 3}` by carrying out the
derivation for

    {x ↦ 3} ⊢ x + 5 ⇓ ???
.....................................................................*)

(* ANSWER: Carrying out each step in the derivation:

    {x ↦ 3} ⊢ x + 5 ⇓
                    | {x ↦ 3} ⊢ x ⇓ 3    (R_var)
                    | {x ↦ 3} ⊢ 5 ⇓ 5    (R_int)
                    ⇓ 8                  (R_+)

   Again, we've labeled each line with the name of the equation that
   was used from the set of equations in Figure 19.1. You should do
   that too. *)

(*.....................................................................
Exercise 3. Carry out the derivation for the semantics of the
expression `let x = 3 in x + 5` in an empty environment. You'll find
the derivation from Exercise 2 to be useful, and you can appeal to it
directly instead of rederiving it.
.....................................................................*)

(* ANSWER:

  {} ⊢ let x = 3 in x + 5 ⇓
                          | {} ⊢ 3 ⇓ 3              (R_int)
                          | {x ↦ 3} ⊢ x + 5 ⇓ 8     (Exercise 2)
                          ⇓ 8                       (R_let)

   Note the labeling of one of the steps with the result from a
   previous exercise.

   The R_let rule specifies that the environment to be used in the
   third line in this derivation should be E{x → v_D}, where
```

```
     * the metavariable E at this point is the empty environment {},
     * the metavariable x is the object variable x
     * the metavariable v_D is 3.

   Extending {} with a mapping of x to 3 gives the environment {x →
   3}, which is exactly the environment that we use in line 3. The
   generation of the extended environment is carried out implicitly;
   the steps in doing so aren't spelled out explicitly here and
   needn't be in your own derivations.  *)

(* Now it's your turn. We recommend doing these exercises with pencil
on paper. Alternatively, you might share a Google doc and work on
developing the solutions there. After you've worked them out and
verified them with staff, you can later copy them into your lab
document. *)

(*.....................................................................
Exercise 4. Carry out the derivation for the semantics of the
expression `x * x` in an environment mapping `x` to `6`, following
the rules in Figure 19.1.
.....................................................................*)

(* ANSWER:

  {x ↦ 6} ⊢ x * x ⇓
                  | {x ↦ 6} ⊢ x ⇓ 6       (R_var)
                  | {x ↦ 6} ⊢ x ⇓ 6       (R_var)
                  ⇓ 36                    (R_* )
 *)

(*.....................................................................
Exercise 5. Carry out the derivation for the semantics of the
expression x - 2 in the environment mapping x to 8, following the
rules in Figure 19.1.
.....................................................................*)

(* ANSWER:

  {x ↦ 8} ⊢ x - 2 ⇓
                  | {x ↦ 8} ⊢ x ⇓ 8       (R_var)
                  | {x ↦ 8} ⊢ 2 ⇓ 2       (R_int)
                  ⇓ 6                     (R_-)
 *)

(*.....................................................................
Exercise 6. Carry out the derivation for the semantics of the
expression (fun x -> x * x) (x - 2) in the environment mapping
x to 8, following the rules in Figure 19.1.
.....................................................................*)

(* ANSWER:

  {x ↦ 8} ⊢ (fun x -> x * x) (x - 2)
```

```
       ⇓
       | {x ↦ 8} ⊢ (fun x -> x * x) ⇓ (fun x -> x * x)   (R_fun)
       | {x ↦ 8} ⊢ x - 2 ⇓ 6                            (Exercise 5)
       | {x ↦ 6} ⊢ x * x ⇓ 36                           (Exercise 4)
       ⇓ 36                                             (R_app)
 *)

(*.....................................................................
Exercise 7. Finally, carry out the derivation for the semantics of the
expression

    let x = 3 + 5 in (fun x -> x * x) (x - 2)

in the empty environment.
.....................................................................*)

(* ANSWER:

    {} ⊢ let x = 3 + 5 in (fun x -> x * x) (x - 2)
           ⇓
           | {} ⊢ 3 + 5 ⇓ 8                               (Exercise 1)
           | {x ↦ 8} ⊢ (fun x -> x * x) (x - 2) ⇓ 36       (Exercise 6)
           ⇓ 36                                           (R_let)
 *)

(*=====================================================================
  Part 2: Pen and paper exercises, dynamic vs. lexical semantics

The derivations you'll be constructing for these exercises can get
*very complicated* -- as long as 33 lines for exercise 12 -- so you'll
want to do them on paper, and check parts as you go.  *)

(*.....................................................................
Exercise 8: For each of the following expressions, derive its final
value using the evaluation rules in Figure 19.1. Show all steps using
pen and paper, and label them with the name of the evaluation rule
used. Where an expression makes use of the evaluation of an earlier
expression, you don't need to rederive the earlier expression's value;
just use it directly. Each expression should be evaluated in an
initially empty environment.

1. 2 * 25

2. let x = 2 * 25 in x + 1

3. let x = 2 in x * x

4. let x = 51 in let x = 124 in x
.....................................................................*)

(* ANSWER:

1.
{} ⊢ 2 * 25 ⇓
```

```
            | {} ⊢ 2  ⇓ 2        (R_int)
            | {} ⊢ 25 ⇓ 25       (R_int)
            ⇓ 50                 (R_* )

2.
{} ⊢ let x = 2 * 25 in x + 1
       ⇓
       | {} ⊢ 2 * 25 ⇓ 50                          (Exercise 8, #1)
       | {x ↦ 50} ⊢ x + 1 ⇓
       |                  | {x ↦ 50} ⊢ x ⇓ 50     (R_var)
       |                  | {x ↦ 50} ⊢ 1 ⇓ 1      (R_int)
       |                  ⇓ 51                     (R_+)
       ⇓ 51                                       (R_let)

3.

{} ⊢ let x = 2 in x * x
       ⇓
       | {} ⊢ 2 ⇓ 2                               (R_int)
       | {x ↦ 2} ⊢ x * x ⇓
       |                | {x ↦ 2} ⊢ x ⇓ 2        (R_var)
       |                | {x ↦ 2} ⊢ x ⇓ 2        (R_var)
       |                ⇓ 4                       (R_* )
       ⇓ 4                                        (R_let)

4.

{} ⊢ let x = 51 in let x = 124 in x
       ⇓
       | {} ⊢ 51 ⇓ 51                             (R_int)
       | {x ↦ 51} ⊢ let x = 124 in x
       |    ⇓
       |    | {x ↦ 51} ⊢ 124 ⇓ 124               (R_int)
       |    | {x ↦ 124} ⊢ x ⇓ 124                (R_var)
       |    ⇓ 124                                 (R_let)
       ⇓ 124                                      (R_let)

 *)

(*.....................................................................
Exercise 9: Evaluate the following expression using the DYNAMIC
environment semantic rules in Figure 19.1. Use an initially empty
environment.

    let x = 2 in
    let f = fun y -> x + y in
    let x = 8 in
    f x
.....................................................................*)

(* ANSWER:

{} ⊢ let x = 2 in let f = fun y -> x + y in let x = 8 in f x
       ⇓
```

```
        | {} ⊢ 2 ⇓ 2                                                          (R_int)
        | {x ↦ 2} ⊢ let f = fun y -> x + y in let x = 8 in f x
        |   ⇓
        |   | {x ↦ 2} ⊢ fun y -> x + y ⇓ fun y -> x + y                      (R_fun)
        |   | {x ↦ 2, f ↦ fun y -> x + y} ⊢ let x = 8 in f x
        |   |   ⇓
        |   |   | {x ↦ 2, f ↦ fun y -> x + y} ⊢ 8 ⇓ 8                      (R_int)
        |   |   | {x ↦ 8, f ↦ fun y -> x + y} ⊢ f x
        |   |   |   ⇓
        |   |   |   | {x ↦ 8, f ↦ fun y -> x + y} ⊢ f ⇓ fun y -> x + y      (R_var)
        |   |   |   | {x ↦ 8, f ↦ fun y -> x + y} ⊢ x ⇓ 8                  (R_var)
        |   |   |   | {x ↦ 8, f ↦ fun y -> x + y, y ↦ 8} ⊢ x + y
        |   |   |   |   ⇓
        |   |   |   |   | {x ↦ 8, f ↦ fun y -> x + y, y ↦ 8} ⊢ x ⇓ 8       (R_var)
        |   |   |   |   | {x ↦ 8, f ↦ fun y -> x + y, y ↦ 8} ⊢ y ⇓ 8       (R_var)
        |   |   |   |   ⇓ 16                                                 (R_+)
        |   |   |   ⇓ 16                                                     (R_app)
        |   |   ⇓ 16                                                         (R_let)
        |   ⇓ 16                                                             (R_let)
        ⇓ 16                                                                 (R_let)

*)

(*.....................................................................
Exercise 10: For each of the following expressions, derive its final
value using the LEXICAL evaluation rules in Figure 19.2. Show all
steps using pen and paper, and label them with the name of the
evaluation rule used. Where an expression makes use of the evaluation
of an earlier expression, you don't need to rederive the earlier
expression's value; just use it directly. Each expression should be
evaluated in an initially empty environment.

1. (fun y -> y + y) 10

2. let f = fun y -> y + y in f 10

3. let x = 2 in let f = fun y -> x + y in f 8

.....................................................................*)
(* ANSWER:

1.
{} ⊢ (fun y -> y + y) 10
       ⇓
       | {} ⊢ (fun y -> y + y) ⇓ [{} ⊢ (fun y -> y + y)]       (R_fun)
       | {} ⊢ 10 ⇓ 10                                          (R_int)
       | {y ↦ 10} ⊢ y + y ⇓
       |                  | {y ↦ 10} ⊢ y ⇓ 10                 (R_var)
       |                  | {y ↦ 10} ⊢ y ⇓ 10                 (R_var)
       |                  ⇓ 20                                  (R_+)
       ⇓ 20                                                    (R_app)

2.
{} ⊢ let f = fun y -> y + y in f 10
```

```
       ⇓
       | {} ⊢ fun y -> y + y ⇓ [{} ⊢ (fun y -> y + y)]          (R_fun)
       | {f ↦ [{} ⊢ (fun y -> y + y)]} ⊢ f 10
       |     ⇓
       |     | {f ↦ [{} ⊢ (fun y -> y + y)]}
       |     |     ⊢ f ⇓ [{} ⊢ (fun y -> y + y)]               (R_var)
       |     | {f ↦ [{} ⊢ (fun y -> y + y)]} ⊢ 10 ⇓ 10        (R_int)
       |     | {y ↦ 10} ⊢ y + y ⇓
       |     |                   | {y ↦ 10} ⊢ y ⇓ 10          (R_var)
       |     |                   | {y ↦ 10} ⊢ y ⇓ 10          (R_var)
       |     |                   ⇓ 20                         (R_+)
       |     ⇓ 20                                            (R_app)
       ⇓ 20                                                   (R_let)

3.
{} ⊢ let x = 2 in let f = fun y -> x + y in f 8
       ⇓
       | {} ⊢ 2 ⇓ 2                                                    (R_int)
       | {x ↦ 2} ⊢ let f = fun y -> x + y in f 8
       |   ⇓
       |   | {x ↦ 2} ⊢ fun y -> x + y ⇓ [{x ↦ 2}, (fun y -> x + y)]   (R_fun)
       |   | {x ↦ 2, f ↦ [{x ↦ 2}, (fun y -> x + y)]} ⊢ f 8
       |   |   ⇓
       |   |   | {x ↦ 2, f ↦ [{x ↦ 2}, (fun y -> x + y)]}
       |   |   |     ⊢ f ⇓ [{x ↦ 2}, (fun y -> x + y)]                 (R_var)
       |   |   | {x ↦ 2, f ↦ [{x ↦ 2}, (fun y -> x + y)]} ⊢ 8 ⇓ 8    (R_int)
       |   |   | {x ↦ 2, y ↦ 8} ⊢ x + y
       |   |   |   ⇓
       |   |   |   | {x ↦ 2, y ↦ 8} ⊢ x ⇓ 2                          (R_var)
       |   |   |   | {x ↦ 2, y ↦ 8} ⊢ y ⇓ 8                          (R_var)
       |   |   |   ⇓ 10                                              (R_+)
       |   |   ⇓ 10                                                  (R_app)
       |   ⇓ 10                                                      (R_let)
       ⇓ 10                                                          (R_let)
*)

(*.....................................................................
Exercise 11: Evaluate the following expression, the same expression as
in Exercise 9, but this time using the LEXICAL environment semantics
rules in Figure 19.2. As before, use an initially empty environment.

    let x = 2 in
    let f = fun y -> x + y in
    let x = 8 in
    f x
.....................................................................*)

(* ANSWER:

{} ⊢ let x = 2 in let f = fun y -> x + y in let x = 8 in f x
       ⇓
       | {} ⊢ 2 ⇓ 2                                                        (R_int)
       | {x ↦ 2} ⊢ let f = fun y -> x + y in let x = 8 in f x
       |   ⇓
```

```
        |   | {x ↦ 2} ⊢ fun y -> x + y ⇓ [{x ↦ 2} ⊢ fun y -> x + y]       (R_fun)
        |   | {x ↦ 2, f ↦ [{x ↦ 2} ⊢ fun y -> x + y]} ⊢ let x = 8 in f x
        |   |   ⇓
        |   |   | {x ↦ 2, f ↦ [{x ↦ 2} ⊢ fun y -> x + y]} ⊢ 8 ⇓ 8        (R_int)
        |   |   | {x ↦ 8, f ↦ [{x ↦ 2} ⊢ fun y -> x + y]} ⊢ f x
        |   |   |   ⇓
        |   |   |   | {x ↦ 8, f ↦ [{x ↦ 2} ⊢ fun y -> x + y]}
        |   |   |   |     ⊢ f ⇓ [{x ↦ 2} ⊢ fun y -> x + y]                (R_var)
        |   |   |   | {x ↦ 8, f ↦ [{x ↦ 2} ⊢ fun y -> x + y]} ⊢ x ⇓ 8    (R_var)
        |   |   |   | {x ↦ 2, y ↦ 8} ⊢ x + y
        |   |   |   |   ⇓
        |   |   |   |   | {x ↦ 2, y ↦ 8} ⊢ x ⇓ 2                          (R_var)
        |   |   |   |   | {x ↦ 2, y ↦ 8} ⊢ y ⇓ 8                          (R_var)
        |   |   |   |   ⇓ 10                                              (R_+)
        |   |   |   ⇓ 10                                                  (R_app)
        |   |   ⇓ 10                                                      (R_let)
        |   ⇓ 10                                                          (R_let)
        ⇓ 10                                                            (R_let)
 *)

(*.....................................................................
Exercise 12: For the following expression, derive its value using the
LEXICAL evaluation rules for imperative programming in Figure 19.4.
Show all steps using pencil and paper, and label them with the name of
the evaluation rule used. The expression should be evaluated in an
initially empty environment and an initially empty store.

    let x = ref 42 in
    (x := !x - 21; !x) + !x ;;

How does your result compare with the value of this expression as
computed by the OCaml interpreter?
.....................................................................*)

(* ANSWER:

{}, {} ⊢ let x = ref 42 in (x := !x - 21; !x) + !x
    ⇓
    | {}, {} ⊢ ref 42
    |   ⇓
    |   | {}, {} ⊢ 42 ⇓ 42, {}                                    (R_int)
    |   ⇓ l1, {l1 ↦ 42}                                           (R_ref)
    | {x ↦ l1}, {l1 ↦ 42} ⊢ (x := !x - 21; !x) + !x
    |   ⇓
    |   | {x ↦ l1}, {l1 ↦ 42} ⊢ x := !x - 21; !x
    |   |   ⇓
    |   |   | {x ↦ l1}, {l1 ↦ 42} ⊢ x := !x - 21
    |   |   |   ⇓
    |   |   |   | {x ↦ l1}, {l1 ↦ 42} ⊢ x ⇓ l1, {l1 ↦ 42}         (R_var)
    |   |   |   | {x ↦ l1}, {l1 ↦ 42} ⊢ !x - 21
    |   |   |   |   ⇓
    |   |   |   |   | {x ↦ l1}, {l1 ↦ 42} ⊢ !x
    |   |   |   |   |   ⇓
    |   |   |   |   |   | {x ↦ l1}, {l1 ↦ 42} ⊢ x ⇓ l1, {l1 ↦ 42} (R_var)
```

```
    |   |   |   |   |   ⇓ 42, {l1 ↦ 42}                                  (R_deref)
    |   |   |   |   | {x ↦ l1}, {l1 ↦ 42} ⊢ 21 ⇓ 21, {l1 ↦ 42}          (R_int)
    |   |   |   |   ⇓ 21, {l1 ↦ 42}                                      (R_-)
    |   |   |   ⇓ (), {l1 ↦ 21}                                         (R_assign)
    |   |   | {x ↦ l1}, {l1 ↦ 21} ⊢ !x
    |   |   |   ⇓
    |   |   |   | {x ↦ l1}, {l1 ↦ 21} ⊢ x ⇓ l1, {l1 ↦ 21}              (R_var)
    |   |   |   ⇓ 21, {l1 ↦ 21}                                          (R_deref)
    |   |   ⇓ 21, {l1 ↦ 21}                                              (R_seq)
    |   | {x ↦ l1}, {l1 ↦ 21} ⊢ !x
    |   |   ⇓
    |   |   | {x ↦ l1}, {l1 ↦ 21} ⊢ x ⇓ l1, {l1 ↦ 21}                  (R_var)
    |   |   ⇓ 21, {l1 ↦ 21}                                              (R_deref)
    |   ⇓ 42, {l1 ↦ 21}                                                 (R_+)
    ⇓ 42, {l1 ↦ 21}                                                    (R_let)
```

This derivation shows the value of the expression as 42. But OCaml evaluates it to 63:

```
    # let x = ref 42 in (x := !x - 21; !x) + !x ;;
    - : int = 63
```

Why is this? The difference arises from the order in which OCaml evaluates the arguments of binary operators. In an expression of the form `P + Q`, OCaml evaluates Q, then P, then computes the sum:

```
    E,S ⊢ P + Q ⇓
                | E, S ⊢ Q ⇓ n, S′
                | E, S′ ⊢ P ⇓ m, S′′
                ⇓ m + n, S′′
```

(or equivalently,

```
    E,S ⊢ P + Q ⇓
                | E, S′ ⊢ P ⇓ m, S′′
                | E, S ⊢ Q ⇓ n, S′
                ⇓ m + n, S′′                    )
```

But the R_+ rule, repeated here for reference, specifies that the evaluation of P and Q occur in the opposite order:

```
    E,S ⊢ P + Q ⇓
                | E, S ⊢ P ⇓ m, S′
                | E, S′ ⊢ Q ⇓ n, S′′
                ⇓ m + n, S′′
```

The difference between these two rules, pertinent only in the presence of side effects, may remind you of the lab puzzle from labs 12 and 13. *)

```
(*=====================================================================
  Part 3: Implementing environments
```

To represent an environment, we need to maintain a mapping from

```
variable names to their values. For simplicity, we will consider only
integer values here. In this part, you'll work on an especially simple
implementation of environments. (Something like this may prove useful
in implementing the final project!) A variable will be represented as
a string, and an environment will be represented as an "association
list" made of pairs of variables and their integer values.

The `List` module has some functions useful for manipulating
association lists. You may want to make use of them here. See
<https://v2.ocaml.org/api/List.html#1_Associationlists>.
 *)

type varid = string ;;
type value = int ;;
type env = (varid * value) list ;;

(*.....................................................................
Exercise 13: Write a function `empty : unit -> env` that returns an
empty environment.
.....................................................................*)

let empty () : env = [] ;;

(*.....................................................................
Exercise 14: Write a function `extend : env -> varid -> value -> env`
that extends an environment; that is, `extend e x v` should return an
environment that maps variables to values just as `e` does *except*
that it maps `x` to `v`. Make sure to handle the case where `x` is
already in the environment.
.....................................................................*)

let extend (e : env) (x : varid) (v : value) : env =
  (x, v) :: (List.remove_assoc x e) ;;

(*.....................................................................
Exercise 15: Write a function `lookup : varid -> env -> value` that
returns the value of a variable in the given environment, raising a
`Not_found` exception if the variable has no value in the environment.
.....................................................................*)

let lookup : varid -> env -> value =
  List.assoc ;;

(* The requested function is already implemented in the `List` library
   as `List.assoc`, including the returning of the `Not_found`
   exception, so we just use it directly here. *)
```